

CSC 452/552 Operations Systems

Project 3 Threads

Name: Amanda Stroud

Bronco ID: 114160989

Date: 12 November 2024

1. Project Overview

This project involved turning into an already designed merge sort and making it thread safe so that several threads can run concurrently with up to 32 threads.

2. Project Management Plan

a) Task 3:

In task 3 I had to make a thread safe merge sort, here I knew I needed to create a new thread for the number of inputted threads, and then pass the parallel merge function into each thread and split the parallel merge values based on the number of threads to be split upon (more information on how I figured things out in reflection).

b) Task 4:

Here I tested the code input each with a different number of threads.

```
stroamanda@DESKTOP-QA0F920:~/OperatingSystems/s452-p1$ ./myprogram 1000 2
3.260986 2
stroamanda@DESKTOP-QA0F920:~/OperatingSystems/s452-p1$ ./myprogram 1000 3
3.990967 3
stroamanda@DESKTOP-QA0F920:~/OperatingSystems/s452-p1$ ./myprogram 1000 4
3.181885 4
stroamanda@DESKTOP-QA0F920:~/OperatingSystems/s452-p1$ ./myprogram 1000 8
2.380127 8
stroamanda@DESKTOP-QA0F920:~/OperatingSystems/s452-p1$ ./myprogram 1000 16
4.386963 16
stroamanda@DESKTOP-QA0F920:~/OperatingSystems/s452-p1$ ./myprogram 1000 32
5.111084 32
```

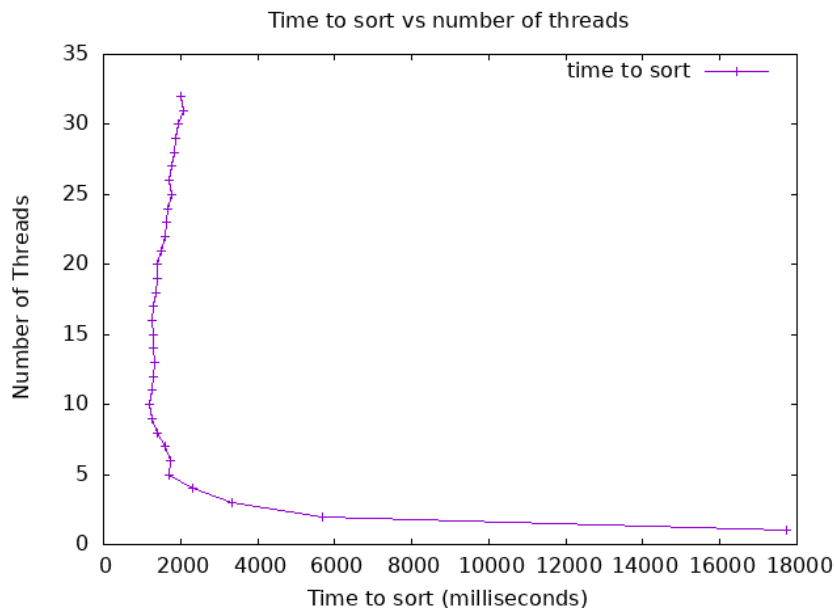
c) Task 5:

For task 5, I attempted to run the script to see if I could see what was missing and I found the problem pretty quickly. I simply had to change what folder of what the “myprogram” pointed to the current directory since I have the script sharing the directory with myprogram.

d) Task 6:

Wrote analysis in analysis.md based on a graph from the thread safe merge sort that I created. It includes the image

of the graph within the file as well answering the questions asked in task 6.



3. Project Deliveries

a) How to compile and use my code?

To compile and use the code, you have to run, clean, make, and make check to view the test results as passing. To run the program you have to run `./myprogram <array_size> <num_threads>` with custom variables and a max number of threads as 32. For the script you have to make sure to do `sudo apt-get install -y gnuplot`, and then it can be ran by doing `./createplot.sh -s 10000000 -f student_plot`

b) Any self-modification?

No self modifications to running or compiling the code, so it should work how it was initially copied.

c) Summary of Results.

The results show the merge sort passing the tests, as well being able to run the program through the command line with custom values and being able to run the bash script and converting the data from the script into a graph.

4. Self-Reflection of Project

This project was pretty interesting and I've started to become more comfortable with including threads into my code. So I first started by copying the typical merge sort into `mergesort_mt` and I looked at the input, it takes an array, the size of the array, and number of threads. So I knew that I couldn't have the actual merge be a part of this

function. Then I saw the `parallel_mergesort` function, so I knew that the merging would be within this function. I then saw it took in an `args` variable, and the function itself says “see struct `parallel_args`”, which I saw as being a struct of variables. So in my parallel merge I took the `args` variable and made it into a parallel struct variable. Then I can just replace the merge values of `A`, `p`, and `r` with `pargs->A`, `pargs->start`, `pargs->end`. Here it works functionally the same as the normal merge sort. Now in the `mergesort_mt` I create a parallel struct array that can be passed through into the parallel merge. I made this an array so it can keep track of the amount of threads at each point based on the start and end value which is why this array is the size of the threads. Now I need a variable to store the value in which the array will be split upon, so I divide the total number of values in the array by the number of threads. Then to create the needed number of threads I created a loop that is designed to create a new thread for the number of threads required. Here I will also fill up the parallel array with the array of values we want to sort, the start index at the current point (we can do the index multiplied by the `splitSize`) and then have the end value be the start value plus the value we are splitting based on, minus 1 to account for having the correct ending indexes to avoid an out of bounds error, then if I am at the last split then the end value is just the array size minus 1.

Now for the creation of the thread I need an array of threads, and then I can input the function I want the thread to be created upon, and the arguments that go into the function, aka the parallel array I had created at a specified index. After that I could just join the threads. I did get stuck a bit on the merging of the sorted threads at the end, but the idea I was working with was I would merge two sorted sections at a time. So once I would merge the two sorted sections this created a “new” section, so then I can compare this new section with the next one that needs to be merged till the end of the list. So the start value will always stay at 0, and then I have an initial end 1 value which is the end in the first section, and then end 2 is the end value of the next section. So now we have values to use in the `merge_s` function. After these values are merged, end1 becomes what end2 was and end2 becomes the next end value of the next section. If end2 ends up greater or equal to the size of the array then we can just set end2 to the size of the array minus 1.

After that everything else was easy to fix up. For the script file, I simply had to change where myprogram was located. Though with the codespace on Github there is a bug that causes the terminal to loop “NOSIGNAL” infinitely when trying to run the program. I’ve had this problem in a few projects but for running the script it causes it to appear randomly while the script is running so it would never be able to finish running without this appearing. So I ended up downloading the code on my computer and to make it easier I used WSL so I was able to get it to run pretty quickly and I didn’t have any more issues.

In terms of sources, I looked at the pthread tutorial for correctly communicating the threads with the parallel merge function.

<https://hpc-tutorials.llnl.gov/posix/>

5. Comments for Project (optional)
6. Use of AI for debugging (optional)